DEVPILOT: AUTONOMOUS AGENTIC SOFTWARE ENGINEERING SYSTEM

System Architecture, Multi-Agent Orchestration & Production Deployment Synopsis

Project: DevPilot (Agents v2)	**Core Frameworks:** LangGraph, LangChain, Groq, Gemini
Author / Owner: ambersaluja20006@gmail.com	**Security & Storage:** Supabase Auth & Cloud Partitions
Repository: lang_graph_ai_project (master)	**Deployment Target:** AWS Amplify / App Runner / Docker

1. Executive Summary

DevPilot is an autonomous, multi-agent AI software engineering framework designed to convert natural language prompts into complete, fully functional, zero-build software applications delivered as standalone packages. Unlike conventional LLM code assistants that output disconnected snippets requiring manual developer compilation, DevPilot manages an end-to-end **Agentic Software Development Life Cycle (SDLC)**.

Powered by **LangGraph**, DevPilot structures AI reasoning into a cyclic, stateful directed graph. The system autonomously classifies intent, inspects project environments, formulates modular architectures, coordinates parallel code generation, validates functionality via test runners, and executes automated self-healing loops when syntax or logic errors occur. Every completed application is rendered in a live sandbox preview and packaged into an instant, zero-build ZIP archive.

2. Problems Addressed & Engineering Solutions

Traditional Challenge	DevPilot Engineering Solution
Context Fragmentation: LLMs generate isolated files that fail to integrate, miss CSS styling, or omit entry points.	**Stateful LangGraph Memory:** Global state synchronizes architectural models and task definitions across every specialized agent node.
Rate Limit (TPM) Exhaustion: Multi-turn agent loops quickly exceed provider ceilings (e.g., Groq 8k TPM).	**Sliding-Window Pruning & Batching:** Coder emits all core files in Turn 1; messages are pruned to retain low-token context with multi-model failover.
Over-Engineered Scaffolding: Heavy build tools (Vite/Webpack) break during compilation without human intervention.	**Zero-Build Standalone Architecture:** Native HTML5, modern CSS3 variables, and vanilla ES6 run directly in any browser out of the box.
Multi-User Data Collisions: Shared workspaces cause concurrent user requests to overwrite each other's files.	**Isolated Cloud Partitions:** Supabase Auth partitions storage under users/{user_id}/, resetting workspace to 0 bytes on each synthesis.

3. Multi-Agent Pipeline Architecture (Agents v2)

DevPilot models software synthesis through specialized, communicating agent personas organized in a LangGraph StateGraph:

Agent Node	Role & Specialization	Core Model & Mechanism
■ **Router**	Analyzes user prompts; routes to CREATE, MODIFY, DEBUG, TEST, or CHAT intent.	Gemini Flash-Lite / Groq Auto

■ **Inspector**	Audits filesystem, languages, dependencies, and project manifests.	Sandboxed Filesystem Tools
■ **Architect**	Designs component layout, data flow, tech stack, and zero-build rules.	Groq GPT-OSS 120B (Structured)
■ **Planner**	Deconstructs architecture into 4–7 prioritized, sequential tasks.	Groq GPT-OSS 120B (Pydantic)
■ **Coder**	Batch-writes source code, modern CSS design systems, and responsive logic.	Groq Tool-Calling + Rule 22 UI
■ **Tester**	Discovers unit tests, executes test commands, and parses stdout/stderr.	Subprocess Runner + LLM Parser
■■ **Debugger**	Investigates test failures and applies surgical fixes in an automated loop.	Groq Reasoning Agent (Max 6x)
■ **Packager**	Compresses user workspace directly from Supabase Storage into ZIP.	Python in-memory zipfile

4. Technical Innovations & Production Upgrades

A. Authentication Gateway & Session Isolation:

- **Enforced Auth Gateway:** Unauthenticated visitors attempting to open /app are instantly intercepted and redirected to /login via an inline <head> guard with zero content flash.
  - **Supabase Cloud Partitions:** Every registered account is granted an isolated storage bucket prefix (users/{user_id}/). On every new prompt, the user's partition is cleanly purged to 0 bytes before generation, ensuring zero cross-run pollution.

B. In-Browser Live Preview & Dynamic Sandbox Cookies:

- **Sub-Resource Scoped Cookies:** Preview requests issue a scoped Set-Cookie: devpilot_user_id={user_id}; Path=/preview/ header. This ensures that relative sub-resource requests (such as style.css or app.js loaded by the iframe) always resolve to the specific user's generated partition instead of falling back to default.
  - **Device Emulation:** Studio dashboard features one-click viewport toggles (Desktop vs. Mobile 375px) and external window launch.

C. Concurrency & Real-Time Antigravity Timers:

- **ThreadingHTTPServer:** Upgraded web server from single-threaded to ThreadingHTTPServer with daemon threads, eliminating socket starvation and blocking during background agent pipeline execution.
  - **Atomic State Initialization:** Server synchronizes is_running = True and start_time before thread dispatch, eliminating early poll race conditions and preventing timer freezes.

5. Cloud Deployment & Containerization Readiness

- **Dockerfile:** Multi-stage, production Python 3.11-slim container with automatic non-root execution and dynamic $PORT binding.
- **Health Probes:** Verified GET /health and GET /api/health endpoints returning HTTP 200 JSON payloads for ALB/Amplify probes.
- **AWS Amplify Build Spec:** amplify.yml pre-configured with caching and container launch scripts.
- **Strict Credential Isolation:** .gitignore and .env.example verify that no API keys or database tokens can leak into version control.

6. Project Status & Summary

DevPilot has achieved stable, production-grade maturity. All automated SDLC stages—from intent extraction to cloud ZIP export—operate reliably with verified sub-35 second synthesis times. The repository is fully staged, tested, and ready for deployment.